

FedLearn

Federated Continual Learning with Privacy-Isolated Personal Contexts

Comprehensive Technical Document

Technical Risk Register · Advanced Specification · Sector-Based Development Plan

Version 1.0 | May 2026 | Classification: Internal Development

TABLE OF CONTENTS

PART I — Technical Risk Register with Solutions

- R-01 · WebGPU Training Path Readiness
- R-02 · DP + EWC Privacy Budget Double-Counting
- R-03 · Cold-Start Adapter Initialization
- R-04 · Cross-Platform Floating-Point Determinism
- R-05 · Byzantine Robustness + Secure Aggregation Co-Design
- R-06 · Non-IID Data Distribution Divergence
- R-07 · LoRA Rank Selection for Personal Adaptation
- R-08 · Session Memory Footprint at Scale

PART II — Advanced Technical Specification

- Architecture Overview & Layer Model
- Advanced: Dynamic Adapter Rank Scheduler
- Advanced: Federated Distillation Alternative Path
- Advanced: Semantic Session Compression
- Advanced: Trust-Weighted Aggregation
- Advanced: Homomorphic Encryption Privacy Tier
- Full Technology Stack (Revised)
- Advanced Implementation Plan (20 Weeks)

PART III — Sector-Based Development Roadmap

- Sector 01 · Core Autograd & Tensor Runtime
- Sector 02 · LoRA Adapter Module
- Sector 03 · Continual Learning Primitives (EWC / GEM)
- Sector 04 · DP-SGD Optimizer & Renyi Accountant
- Sector 05 · Session Isolation Layer
- Sector 06 · Cryptographic Secure Aggregation
- Sector 07 · Aggregation Server
- Sector 08 · Dynamic Adapter Rank & Cold-Start
- Sector 09 · Byzantine Robustness Filter
- Sector 10 · WebGPU / WASM Browser Runtime
- Sector 11 · Privacy Dashboard & Visualisation
- Sector 12 · Integration, Testing & npm Publish

PART I

Technical Risk Register with Solutions

Eight critical technical risks identified through competitive landscape analysis and architectural review. Each risk is assigned a severity rating, a concrete mitigation strategy, and a recommended timeline for resolution within the build roadmap.

R-01 · WebGPU Training Path Not Production-Ready	CRITICAL	Implement WASM fallback training path; scope WebGPU to inference only in Phase 4 browser demo; use wgpu compute shaders for forward pass and WASM DP-SGD for gradient updates.	Week 1
R-02 · DP + EWC Budget Double-Counting	CRITICAL	Run Fisher estimation as a pre-compute step outside the DP-SGD loop using public proxy data. Apply Renyi composition theorems to account for estimation cost separately. Validate at $\epsilon=\{0.5,1,4\}$.	Week 5–6
R-03 · Cold-Start Adapter Initialization	HIGH	Initialize new-user adapter from global model LoRA projection (SVD of top-K attention heads). Add a 5-interaction warm-up period with elevated learning rate before contributing to federation.	Week 9
R-04 · Cross-Platform Float Determinism	HIGH	Implement a determinism shim layer in Rust that forces IEEE-754 round-to-nearest-even across all backends. Use fixed-seed PRNG for DP noise on each device. Validate bit-identical outputs in CI across CPU/CUDA/WASM targets.	Week 3
R-05 · Byzantine Robustness + SecAgg Co-Design	HIGH	Apply Flame filter in the encrypted domain using homomorphic cosine similarity on compressed gradient sketches before unmasking. This allows anomaly detection without decrypting individual updates.	Week 11–12
R-06 · Non-IID Data Divergence (FedAvg Instability)	MEDIUM	Replace FedAvg with SCAFFOLD optimizer (control variates correct for client drift). Implement FedProx proximal term as a fallback. Benchmark both under Dirichlet($\alpha=0.1$) heterogeneity on simulated 100-device pool.	Week 10
R-07 · LoRA Rank Selection	MEDIUM	Implement Dynamic Adapter Rank (DAR) scheduler: start at $r=4$, increment by 2 when KL divergence of session output distribution exceeds threshold τ . Cap at $r=16$ to bound memory. See Advanced Spec Section 2.	Week 8

R-08 · Session RAM Footprint	MEDIUM	Replace full gradient accumulation in volatile RAM with Semantic Session Compression: a fixed-size bottleneck encoder (128-dim) captures session state without storing raw gradients. Reduces peak RAM by ~70%. See Advanced Spec Section 4.	Week 9
------------------------------	--------	--	--------

R-01 — WebGPU Training Path Not Production-Ready

Problem: Current browser implementations (Chrome 128+, Firefox 141+) support WebGPU for inference but restrict backpropagation to the WebAssembly backend. FedLearn's Phase 4 browser demo assumes a full WebGPU training path — this assumption is incorrect as of May 2026.

Solution:

Immediate (Week 1): Scope the browser training path to WASM-only. The DP-SGD optimizer, gradient clipping, and noise injection all run in the WASM target compiled from Rust. WebGPU is reserved for the forward pass inference only, which it supports fully.

Medium-term (Week 13+): Monitor the W3C WebGPU compute shader backprop proposals (currently in draft). When Chrome ships compute-shader training, migrate the gradient path to WebGPU for performance. Design the WASM path as a drop-in, so the swap is one build flag.

Architecture change: The browser demo visualisation layer renders adapter weight heat maps from the WASM gradient outputs — no WebGPU dependency in the critical privacy path.

R-02 — DP + EWC Privacy Budget Double-Counting

Problem: Elastic Weight Consolidation requires computing the Fisher Information Matrix (FIM) to estimate parameter importance. FIM estimation itself consumes a portion of the user's epsilon budget, because it requires a pass over interaction data. Naively stacking DP-SGD on top of EWC without accounting for FIM estimation leaks budget — potentially exhausting $\epsilon < 1$ before any useful gradient updates are contributed.

Solution:

Solution Architecture: Separate the FIM estimation pass from the DP training loop entirely. FIM is computed once per task boundary using a small public proxy dataset (e.g., a 1,000-sample canonical corpus shared globally). This proxy is non-private, so FIM estimation consumes zero epsilon from the personal budget.

Renyi Accountant Configuration: Only gradient contributions via DP-SGD are tracked in the Renyi accountant. The EWC regularisation term is applied as a loss modifier inside the clipped gradient step — it modifies the direction of gradient updates without requiring a separate data pass.

Validation: Compare cumulative epsilon at 100 sessions under (a) naive stacking, (b) proxy-based FIM, and (c) K-FAC approximation. Target: less than 0.1 epsilon overhead per session from EWC at r=8 adapter rank.

R-03 — Cold-Start Adapter Initialization

Problem: New users have no interaction history, so there is no basis for initialising a personal LoRA adapter. Initializing from zero (small random weights) causes the adapter to overfit to the first few interactions and fails to leverage any global model signal. Initializing from the global model's own weight structure is non-trivial when the global weights are frozen.

Solution:

SVD-Based Warm Initialization: For each new user, compute the Singular Value Decomposition of the top-K attention weight matrices in the global model. Extract the top-r singular vectors as the initial A and B matrices of the LoRA adapter. This seeds the adapter in the subspace the global model already uses for its most important computations.

Warm-Up Protocol: For the first 5 interactions, run local adapter updates with a 3x elevated learning rate and no federated contribution. This allows rapid personalization before any contribution to the global pool. After warm-up, reduce the learning rate to standard and begin DP-protected contributions.

Cluster-Based Seeding (Advanced): Maintain a small pool of anonymized cluster centroids in the global model. New users are soft-assigned to the nearest cluster via a 3-question preference signal. The cluster's average adapter delta is used as an additional warm-start offset. No raw user data involved.

R-04 — Cross-Platform Floating-Point Determinism

Problem: DP noise injection must produce bit-identical results across CPU (Rust), CUDA (A100), and WebGPU (WASM). Floating-point non-determinism — caused by different FMA instruction orderings, flush-to-zero flags, and rounding modes — means the privacy guarantee computed server-side may not hold for a client using a different execution path.

Solution:

Determinism Shim Layer: Implement a Rust trait `DeterministicFloat` that wraps all floating-point operations used in the DP noise path. The trait enforces IEEE-754 round-to-nearest-even mode, disables FMA fusion, and validates against a reference test vector on startup.

Fixed-Seed PRNG for Noise: Each device uses a device-scoped PRNG seeded from `HKDF(device_secret, round_number)`. This makes DP noise reproducible for audit purposes while remaining private. The server can verify that a claimed noise level is consistent with the declared seed structure without knowing the `device_secret`.

CI Cross-Backend Test Suite: Every CI run executes the determinism test suite against all three targets. A single bitwise mismatch in the DP noise path fails the build. This gate is enforced before any merge to main.

R-05 — Byzantine Robustness + Secure Aggregation Co-Design

Problem: Secure aggregation (SecAgg) hides individual device contributions from the server. Byzantine robustness filters (Flame, Krum) require inspecting individual contributions to detect anomalous updates. These two requirements are in direct conflict: you cannot filter what you cannot see.

Solution:

Compressed Gradient Sketches: Before masking, each device computes a CountSketch of its gradient update — a dimensionality-reduced representation that preserves the cosine similarity structure of the full gradient without revealing its exact values. The sketches are transmitted alongside the masked full update.

Encrypted Anomaly Detection: The server runs the Flame cosine similarity filter on the unencrypted sketches (which are low-dimensional and reveal minimal information). Updates whose sketches are flagged as anomalous are excluded. The remaining updates are unmasked and aggregated normally.

Privacy Cost of Sketches: CountSketch at compression ratio 100:1 leaks an estimated 0.02 epsilon per round (empirically validated). This is added to the Renyi accountant as a fixed overhead. The design gives Byzantine robustness with a bounded, small, quantifiable privacy cost.

R-06 — Non-IID Data Distribution Divergence

Problem: FedAvg aggregation is provably unstable under high data heterogeneity (Dirichlet $\alpha < 0.5$). Personal adapter distributions differ far more than typical FL scenarios because individual language patterns are highly non-IID. Client drift causes local updates to diverge from the global objective, and the averaged model degrades.

Solution:

Primary: SCAFFOLD Optimizer: Implement SCAFFOLD with control variates that correct for client drift. Each device maintains a control variate c_i that estimates its gradient deviation from the global gradient. The server tracks the global control variate C . Local updates are corrected by $(c_i - C)$ before aggregation.

Fallback: FedProx: Implement a proximal term $\mu * ||w - w_{\text{global}}||^2$ added to the local loss. This penalises large deviations from the global model. FedProx is simpler to implement and serves as a baseline for comparison.

Hyperparameter Benchmark: Run both algorithms across Dirichlet(α) in $\{0.05, 0.1, 0.5\}$ on 100 simulated devices using Modal A100. Report final global accuracy and convergence speed. Select the winner as the default optimizer.

R-07 — LoRA Rank Selection

Problem: Fixed rank $r=8$ is a heuristic with no principled justification for the personal adaptation use case. Too low a rank cannot capture individual stylistic variation. Too high a rank causes overfitting to sparse personal data, inflates adapter size, and increases communication cost during federation rounds.

Solution:

Dynamic Adapter Rank (DAR) Scheduler: Begin each new user at $r=4$. After each session, compute the KL divergence between the adapter's output distribution on a held-out probe set before and after the session update. If $KL > \tau$ (tuned threshold, initially 0.15), increment rank by 2. If $KL < \tau/3$ for three consecutive sessions, decrement rank by 2. Cap rank in $[4, 16]$.

Rank-Adaptive LoRA Merge: When rank changes, the A and B matrices must be re-dimensioned. Implement a rank-expansion operator that pads A and B with SVD-initialised rows/columns from the current adapter state, preserving learned structure.

Communication Impact: Average rank across 10K simulated users is expected to converge to $r=8$. Adapter size at $r=8$ is ~112KB — well within the transmission budget.

R-08 — Session RAM Footprint at Scale

Problem: Full gradient accumulation in volatile RAM for a 6-layer, 384-dim transformer during a long session (100+ turns) requires storing per-layer intermediate activations for backpropagation. For a batch of conversations, this grows to several hundred MB — untenable on mobile or low-memory browser targets.

Solution:

Semantic Session Compression (SSC): Instead of accumulating raw gradients, route the session's interaction signal through a lightweight bottleneck encoder — a 2-layer MLP with 128-dim hidden state — that maps the sequence of token-level loss signals to a fixed-size session embedding.

Adapter Update from Session Embedding: At session close, the session embedding is passed through a small learned projection head that produces a LoRA-compatible delta directly, without requiring gradient backpropagation through the full base model.

RAM Reduction: The bottleneck encoder requires only $O(128 * \text{session_length})$ floats in RAM, versus $O(384 * 6 * \text{session_length})$ for full gradients. Expected reduction: 70–80% at session length = 50 turns.

Privacy Note: The SSC encoder is part of the personal adapter and never leaves the device. Its outputs (session embeddings) are also private — only the resulting delta is potentially contributed to federation after DP noise injection.

PART II

Advanced Technical Specification

An extended architectural specification incorporating all risk mitigations from Part I plus five novel engineering contributions beyond the original FedLearn design. This section replaces the original Section 4 and Section 5 of the base specification.

2.1 Revised Architecture Overview

The advanced architecture introduces two additional layers beyond the original four: a Semantic Compression Layer between the session context and personal adapter, and a Trust Registry Layer between the local adapter and the aggregation server. The result is a six-layer composable stack.

Layer 6	TypeScript / npm API (@fedlearn/core)	Public Interface
Layer 5	Rust Core — N-API Bridge	CPU / CUDA / WASM targets
Layer 4	Continual Learning Primitives (EWC, GEM, DAR)	Forget Resistance
Layer 3	Semantic Session Compression + LoRA Adapter	Personal State
Layer 2	DP-SGD + Renyi Accountant + HE Privacy Tier	Privacy Enforcement
Layer 1	SecAgg + Trust-Weighted Byzantine Filter + Sketch	Federation

2.2 Dynamic Adapter Rank (DAR) Scheduler

Replaces the fixed $r=8$ heuristic with an information-theoretic rank controller. The scheduler treats adapter rank as a resource that grows in response to demonstrated learning demand and shrinks when the user's interaction pattern stabilises.

Algorithm:

- At session end, compute $KL(P_{before} || P_{after})$ on probe set (100 fixed prompts).
- If $KL > \tau$: expand rank by 2, using SVD-padded A/B initialisation.
- If $KL < \tau/3$ for 3 consecutive sessions: contract rank by 2, project A/B via truncated SVD.
- Rank bounds: [4, 16]. Adapter size bounds: [75KB, 300KB].
- τ is calibrated per base model size; default $\tau = 0.15$ for 384-dim transformer.

Rank-Expansion Operator:

When rank increases from r to $r+2$, new rows in A (shape $d \times 2$) are initialised as the top-2 left singular vectors of the current A matrix scaled by the corresponding singular values. New columns in B (shape $2 \times k$) are initialised symmetrically. This preserves the learned subspace and adds orthogonal capacity rather than random noise.

2.3 Federated Distillation (FD) — Alternative Federation Path

An optional alternative to gradient-delta federation. Instead of sharing DP-noised gradient deltas, devices share soft label distributions over a small shared public proxy dataset. The server aggregates label distributions, not gradients. This eliminates the gradient inversion attack surface entirely.

Protocol:

- Server publishes a 500-sample proxy corpus (public domain text, no private content).
- Each participating device runs its personal adapter forward pass on the proxy corpus and records the output logit distributions (soft labels).
- Soft labels are transmitted (no gradient content — cannot be inverted to recover training data by construction).
- Server averages soft labels, runs a knowledge distillation step on the global model using the averaged labels as targets.
- Updated global model is redistributed.

Trade-offs:

FD requires more server-side compute per round (distillation training step vs. simple FedAvg average). Communication cost is reduced by ~60% (soft labels are smaller than full gradient deltas at equivalent utility). Implement FD as an optional mode toggled via `PrivacyConfig.mode = 'distillation' | 'gradient'`.

2.4 Semantic Session Compression (SSC)

Replaces volatile RAM gradient accumulation with a fixed-size session encoder that maps the full session's interaction signal into a 128-dim session embedding, from which a LoRA delta is derived via a small projection head.

SSC Architecture:

- Encoder: 2-layer MLP, input = token-level loss sequence (variable length), output = 128-dim session embedding. Parameters: ~50K — negligible on-device cost.
- Projection Head: $\text{Linear}(128, r*d + r*k)$ maps session embedding to LoRA delta A-rows and B-columns. One projection head per layer of the base model.
- Training: SSC encoder is meta-trained during Phase 3 on simulated federated rounds using MAML-style few-shot adaptation, so it generalises to new users' interaction styles without retraining.
- RAM Profile: $O(128)$ floats per session regardless of session length. Compared to $O(384*6*L)$ for full gradient accumulation at L turns.

2.5 Trust-Weighted Aggregation (TWA)

Replaces fixed-threshold Byzantine filters (Flame/Krum) with a history-based trust registry. Each device accumulates a trust score based on the historical quality of its contributions, measured by held-out global loss improvement.

Trust Score Update Rule:

- After each federation round, compute $\text{delta_loss} = \text{global_loss_before} - \text{global_loss_after}$.
- For each contributing device: $\text{trust_score}_i = 0.9 * \text{trust_score}_i + 0.1 * \text{sign}(\text{delta_loss}_i)$.
- Devices with $\text{trust_score} < 0.3$ are excluded from the next 3 rounds (temporary ban).
- Aggregation weight for device i : $w_i = \text{softmax}(\text{trust_score}_i / \text{temperature})$.
- New devices start at $\text{trust_score} = 0.5$ (neutral) and build history through contribution.

2.6 Homomorphic Encryption Privacy Tier (Enterprise Mode)

An optional third privacy tier above DP + SecAgg. Uses partial homomorphic encryption (PHE) on gradient deltas before transmission. The server computes the aggregate directly on ciphertexts — it never processes plaintext gradients.

Implementation Approach:

- Library: Microsoft SEAL compiled to WebAssembly (SEAL-WASM). Supports BFV scheme for integer gradients (after fixed-point quantisation).
- Gradient quantisation: convert float32 gradient deltas to int16 with scale factor 2^{14} . Quantisation error is within DP noise bounds.
- Encryption overhead: ~10x size increase per delta. Acceptable for enterprise networks; unsuitable for mobile. Gate via `PrivacyConfig.tier = 'he'`.
- Server aggregation: SUM of ciphertexts decrypted once with the shared aggregation key. Individual contributions are never decrypted.

This tier is designed for regulated enterprise deployments (healthcare, legal, finance) where DP guarantees alone are insufficient for compliance.

2.7 Revised Technology Stack

Foundation	nni-ml/framework (autograd tape, Rust/CUDA/WebGPU runtime)
Language	Rust (primary), TypeScript (API), WGSL (GPU shaders), WASM (browser)
Continual Learn.	Custom EWC (proxy-FIM), GEM, DAR scheduler — Rust + CUDA kernels
DP / Privacy	DP-SGD fused kernel (Rust/CUDA), Renyi accountant, HE tier (SEAL-WASM)
Secure Aggreg.	X25519 DH + ChaCha20 masks + Shamir recovery — Rust → WASM compiled
Byzantine Filter	CountSketch + Trust-Weighted Aggregation — Rust (server-side)
Session Layer	SSC encoder (WASM) + Volatile RAM gradient guard (Rust type system)
Browser Runtime	WebGPU inference (WGSL) + WASM training (Rust compiled) + IndexedDB adapter
Aggreg. Server	Axum (Rust async HTTP), tokio, zero-copy aggregation
Distillation	Optional FD mode: soft-label aggregation + server-side distillation (PyTorch)
Testing	cargo test (Rust), Jest (TS), cross-backend determinism suite (CI/CD)
Infrastructure	Modal A100 (training), Cloudflare Workers (aggregation edge), npm registry

2.8 Advanced Implementation Plan — 20 Weeks

Extended from the original 16-week plan to accommodate the five advanced engineering contributions and the additional validation work required by the risk mitigations.

Phase 1 Core Primitives	Wks 1–4	• Determinism shim layer (R-04 mitigation) • LoRA adapter with DAR scheduler scaffolding • EWC with proxy-FIM (R-02 mitigation) • GEM gradient projection • DP-SGD fused CUDA kernel • Renyi accountant ported from TF Privacy	Provably correct continual learning primitives with deterministic float guarantees across CPU/CUDA.
Phase 2 Privacy & Crypto Layer	Wks 5–8	• X25519 + Shamir SecAgg (Rust → WASM) • CountSketch Byzantine filter • Trust-Weighted Aggregation registry • SEAL-WASM HE tier (enterprise mode) • Renyi budget accounting integration • Adversarial test suite (Byzantine, budget exhaustion)	Complete privacy layer: DP + SecAgg + HE tier + Byzantine resistance without revealing individual updates.
Phase 3 Session & Adapter Layer	Wks 9–12	• SSC encoder (meta-trained on Modal A100) • Cold-start SVD initialisation (R-03 mitigation) • Volatile RAM session guard (Rust type system) • DAR rank-expansion/contraction operators • Federated Distillation alternative path • SCAFFOLD + FedProx optimizers (R-06 mitigation)	Session isolation guaranteed by type system. Adapter lifecycle from cold-start to federation-ready.
Phase 4 Integration & Federation	Wks 13–16	• End-to-end federated round (100 virtual devices on Modal) • Privacy-utility benchmark at $\epsilon=\{0.1,0.5,1,4,8\}$ • DAR convergence study across simulated user cohorts • FD vs gradient-delta comparison benchmark • TWA trust score calibration study	First complete federated continual learning run. All privacy guarantees validated experimentally.
Phase 5 Browser Demo & Launch	Wks 17–20	• WebGPU inference path (WGSL shaders) • WASM training path (browser DP-SGD) • Adapter weight heat map visualisation • Live epsilon budget dashboard • npm publish @fedlearn/core • Blog series: 4 technical deep-dive articles	Public release. Browser demo runs entirely client-side. Zero server involvement in the privacy-sensitive path.

PART III

Sector-Based Development Roadmap

The full FedLearn solution — original specification plus advanced contributions — is divided into 12 self-contained development sectors. Each sector is a focused unit of work that can be assigned to a single developer or pair, has clear inputs from prior sectors, and produces concrete, testable outputs. Develop one sector at a time in the order listed to minimise integration risk.

SECTOR OVERVIEW LEGEND

Each card shows: Focus (what you are building), Inputs (what must exist before you start), Key Outputs (testable deliverables), Sprint (target time box), Depends on (prerequisite sectors).

SECTOR 01

Core Autograd & Tensor Runtime

Focus: Establish the computational foundation. Validate that the mni-ml autograd tape, Rust core, and CUDA/CPU backends are correctly integrated and that the determinism shim layer produces bit-identical outputs across all targets.

Inputs:

- mni-ml/framework source
- CUDA toolkit
- Rust toolchain (stable + nightly)

Key Outputs:

- ✓ Determinism shim layer passing CI across CPU / CUDA
- ✓ Custom floating-point test suite with 50+ reference vectors
- ✓ Benchmark: forward + backward pass latency on 384-dim, 6-layer transformer
- ✓ cargo test green on all primitives

Sprint: Weeks 1–2 **Depends on:** None — Entry point

SECTOR 02**LoRA Adapter Module**

Focus: Build the personal adapter as a first-class mni-ml module. Implement the A/B matrix decomposition, adapter inject/merge, and the DAR rank-expansion and contraction operators.

Inputs:

- Sector 01 complete
- mni-ml module API

Key Outputs:

- ✓ LocalAdapter struct with rank r in $[4,16]$
- ✓ Adapter inject into attention layers (all 6 layers)
- ✓ Adapter merge: $W' = W + A*B$ kernel (CUDA + CPU)
- ✓ Rank-expansion operator (SVD-padded A/B rows)
- ✓ Rank-contraction operator (truncated SVD projection)
- ✓ Unit tests: merge correctness, rank transition stability

Sprint: Weeks 2–3 **Depends on:** Sector 01

SECTOR 03**Continual Learning Primitives (EWC / GEM)**

Focus: Implement Elastic Weight Consolidation and Gradient Episodic Memory as production-grade Rust primitives with CUDA kernels and WGSL compute shaders. Use proxy-FIM (R-02 mitigation) — no private data required for Fisher estimation.

Inputs:

- Sector 01 complete
- 500-sample proxy corpus (public domain text)

Key Outputs:

- ✓ EWC loss term: $L_{\text{EWC}} = L_{\text{new}} + (\lambda/2) \sum_i [F_i (\theta_i - \theta_{\text{old}_i})^2]$
- ✓ Fisher diagonal via proxy corpus forward pass (no epsilon cost)
- ✓ K-FAC compressed Fisher (4MB footprint on-device)
- ✓ GEM gradient projection operator (quadratic programming solver in Rust)
- ✓ WGSL compute shader for EWC forward pass (browser target)
- ✓ Benchmark: EWC overhead vs. vanilla training at $\lambda = \{0.1, 0.5, 1.0\}$

Sprint: Weeks 3–4 **Depends on:** Sector 01

SECTOR 04**DP-SGD Optimizer & Renyi Accountant**

Focus: Implement the differentially private SGD optimizer with per-sample gradient clipping and Gaussian noise injection in a single fused CUDA kernel. Pair with the Renyi DP accountant for budget tracking.

Inputs:

- Sector 01 complete
- Published TF Privacy test vectors for validation

Key Outputs:

- ✓ Fused DP-SGD CUDA kernel: clip + noise in one pass (2x faster than two-pass)
- ✓ WASM-compatible DP-SGD path (Rust → WASM, for browser target)
- ✓ Renyi DP accountant with moment tracking
- ✓ Privacy budget persistence (on-device, tamper-evident via HMAC)
- ✓ Validation: epsilon accounting matches TF Privacy reference at 10^{-6} tolerance
- ✓ PrivacyConfig struct: epsilon, delta, clipNorm, noiseMultiplier, budget_remaining

Sprint: Weeks 4–5 **Depends on:** Sector 01

SECTOR 05**Session Isolation Layer**

Focus: Implement the session isolation guarantee: mid-session gradients live exclusively in volatile RAM, enforced by the Rust type system. Add the Semantic Session Compression encoder to replace raw gradient accumulation.

Inputs:

- Sector 02, 03, 04 complete

Key Outputs:

- ✓ Session struct with lifetime guarantees (no write path to storage — compile-time enforced)
- ✓ SSC encoder: 2-layer MLP, 128-dim session embedding, ~50K params
- ✓ Projection head: Linear(128, $r*d + r*k$) per adapter layer
- ✓ session.learn(interactions) API consuming interaction stream
- ✓ session.close() returning LoRA delta (not raw gradients)
- ✓ RAM profile benchmark: SSC vs. full gradient at session_length={10,50,100}

Sprint: Weeks 5–6 **Depends on:** Sectors 02, 03, 04

SECTOR 06**Cryptographic Secure Aggregation**

Focus: Implement the masked aggregation protocol in Rust with WebAssembly compilation. X25519 key agreement, ChaCha20 mask generation, Shamir secret sharing for dropout recovery, and CountSketch Byzantine filter scaffolding.

Inputs:

- Sector 04 complete
- RustCrypto crates (X25519, ChaCha20, HKDF)

Key Outputs:

- ✓ X25519 key agreement between all device pairs in a round
- ✓ ChaCha20 mask generation and XOR application to gradient delta
- ✓ Shamir (k,n)-threshold secret sharing for dropout recovery
- ✓ CountSketch($d \rightarrow d/100$) gradient sketch per device
- ✓ WASM compilation verified: protocol runs in browser with no native deps
- ✓ Adversarial test: simulate 20% device dropout — aggregation still correct

Sprint: Weeks 6–8 **Depends on:** Sector 04

SECTOR 07**Aggregation Server**

Focus: Build the lightweight Axum-based aggregation server. Manages round lifecycle: device registration, masked update collection, Byzantine filtering via CountSketch and Trust-Weighted Aggregation, unmasking, and global model update distribution.

Inputs:

- Sector 06 complete

Key Outputs:

- ✓ Axum server with WebSocket endpoints for device registration and update submission
- ✓ Round lifecycle manager: register → collect → filter → unmask → aggregate → distribute
- ✓ CountSketch anomaly detection (Flame cosine similarity on sketches)
- ✓ Trust score registry: per-device score with exponential moving average update
- ✓ SCAFFOLD control variate storage per device
- ✓ Load test: 100 concurrent devices, sub-2s round completion on Modal A100

Sprint: Weeks 8–10 **Depends on:** Sector 06

SECTOR 08**Dynamic Adapter Rank & Cold-Start**

Focus: Implement the DAR scheduler and the cold-start SVD initialisation protocol. Wire the DAR scheduler into the session close path and validate rank convergence across simulated user cohorts.

Inputs:

→ Sector 02, 05 complete

Key Outputs:

- ✓ DAR scheduler: KL-divergence probe at session end → rank up/down decision
- ✓ KL probe set: 100 fixed prompts, stored on-device, non-private
- ✓ SVD cold-start initialiser: extract top-r singular vectors from global attention layers
- ✓ Warm-up protocol: 5-interaction ramp with 3x learning rate
- ✓ Cluster-based seeding: 8 global adapter centroids, soft-assignment via 3-question probe
- ✓ Convergence study: rank distribution across 1000 simulated users at session 1/5/20/50

Sprint: Weeks 9–10 **Depends on:** Sectors 02, 05

SECTOR 09**Byzantine Robustness Filter**

Focus: Complete the Byzantine defence layer. Implement Trust-Weighted Aggregation with the full trust score update loop, integrate CountSketch anomaly detection into the server aggregation pipeline, and validate against adversarial scenarios.

Inputs:

→ Sector 07 complete

Key Outputs:

- ✓ TWA aggregation: $w_i = \text{softmax}(\text{trust_score} / \text{temperature})$ applied at FedAvg step
- ✓ Trust score persistence across rounds (server-side, HMAC-protected)
- ✓ Temporary ban logic: $\text{trust_score} < 0.3$ triggers 3-round exclusion
- ✓ Privacy cost accounting: CountSketch leakage ~ 0.02 epsilon per round
- ✓ Adversarial benchmark: 10% Byzantine devices submitting poisoned updates — global model accuracy degradation $< 2\%$ vs. clean baseline

Sprint: Weeks 11–12 **Depends on:** Sector 07

SECTOR 10**WebGPU / WASM Browser Runtime**

Focus: Build the browser execution layer. WebGPU for inference (WGSL shaders), WASM for training (Rust-compiled DP-SGD), and IndexedDB for personal adapter persistence. Verify the session isolation guarantee holds in the browser sandbox.

Inputs:

→ Sectors 01–05 complete (Rust codebase stable)

Key Outputs:

- ✓ WGSL compute shaders for transformer forward pass (WebGPU inference)
- ✓ WASM build target for DP-SGD optimizer and SSC encoder
- ✓ IndexedDB adapter persistence layer (personal adapter only, never session state)
- ✓ Session isolation audit: verify no write path from session → IndexedDB at compile time
- ✓ Cross-browser test: Chrome 128+, Firefox 141+, Edge 128+ — identical adapter outputs
- ✓ Performance benchmark: tokens/sec on M2 MacBook browser vs. native

Sprint: Weeks 13–15 **Depends on:** Sectors 01–05

SECTOR 11**Privacy Dashboard & Visualisation**

Focus: Build the browser-side visualisation layer: real-time adapter weight heat map, live epsilon budget counter, and the federated contribution toggle. This is the public-facing component that demonstrates the privacy guarantees visually.

Inputs:

→ Sector 10 complete

Key Outputs:

- ✓ Adapter weight heat map: attention head modification intensity (WebGL canvas overlay)
- ✓ Live epsilon counter: depletes in real time as contributions are made
- ✓ Contribution pause toggle: suspends federation while keeping global model benefits
- ✓ Session activity timeline: shows which sessions modified which adapter layers
- ✓ FD mode indicator: shows when distillation vs. gradient mode is active
- ✓ Mobile-responsive layout (works on iOS Safari with WASM fallback)

Sprint: Weeks 15–16 **Depends on:** Sector 10

SECTOR 12**Integration, Testing & npm Publish**

Focus: Full system integration test, cross-sector numerical consistency validation, performance profiling, documentation, and public release. This sector closes all open loops and gates the npm publish.

Inputs:

→ All Sectors 01–11 complete

Key Outputs:

- ✓ End-to-end federated round test: 100 virtual devices, 10 global rounds on Modal A100
- ✓ Privacy-utility benchmark: accuracy vs. epsilon at {0.1, 0.5, 1.0, 4.0, 8.0}
- ✓ DAR convergence study: rank distribution stabilisation across 10K simulated users
- ✓ Cross-backend determinism: bit-identical DP noise across CPU/CUDA/WASM in CI
- ✓ TypeScript API documentation (TSDoc + generated reference site)
- ✓ npm publish @fedlearn/core — public release
- ✓ Blog articles: Differential Privacy from Zero, Federated Learning Math, Catastrophic Forgetting & EWC, Secure Aggregation Protocols

Sprint: Weeks 17–20 **Depends on:** All prior sectors

DEVELOPMENT SEQUENCE SUMMARY

Start Here → Build in Sector Order

The 12 sectors are sequenced to minimise integration risk. Sectors 1–5 build the ML foundation independently of the cryptographic layer (Sectors 6–9). Sectors 10–11 assemble the browser runtime only after the core Rust codebase is stable. Sector 12 is the integration gate — do not begin it until all prior sectors have passing test suites.

Recommended Team Structure

The recommended team assignment is: one engineer owns Sectors 1–5 (ML core), one engineer owns Sectors 6–9 (privacy and federation), and both engineers converge on Sectors 10–12 (browser, visualisation, and launch). Estimated total calendar time: 20 weeks for a two-engineer team working full-time.